



Algorithms for the bin packing problem with scenarios

Yulle G. F. Borges¹ · Vinícius L. de Lima¹ · Flávio K. Miyazawa¹ ·
Lehilton L. C. Pedrosa¹ · Thiago A. de Queiroz² · Rafael C. S. Schouery¹

Accepted: 5 October 2024 / Published online: 24 October 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

This paper presents theoretical and practical results for the bin packing problem with scenarios, a generalization of the classical bin packing problem which considers the presence of uncertain scenarios, of which only one is realized. For this problem, we propose approximation algorithms whose ratios are bounded by the square root of the number of scenarios times the approximation ratio for an algorithm for the vector bin packing problem. We also show how an asymptotic polynomial-time approximation scheme is derived when the number of scenarios is constant, that is, not a part of the input. As a practical study of the problem, we present a branch-and-price algorithm to solve an exponential set-cover model and a variable neighborhood search heuristic. Experiments show the competence of the branch-and-price in obtaining optimal solutions for about 59% of the instances considered, while the combined heuristic and branch-and-price optimally solved 62% of the instances considered.

Keywords Bin packing problem · Scenarios · Approximation algorithm · Variable neighborhood search · Branch-and-price algorithm

1 Introduction

Cutting and packing problems have been widely studied in the context of Operations Research, mainly because of their properties and real-world applicability (Wäscher et al. 2007). Among these problems, the *Bin Packing Problem* (BPP) asks to pack a set of one-dimensional items, each of a given size, into the least possible number of bins of given identical capacities, where the total size of items in a bin does not exceed the bin capacity. Several variants of this problem have been studied in the literature, either due to their theoretical interest, or their high applicability in different industries (Sweeney and Paternoster 1992; Dahmani et al. 2013).

✉ Rafael C. S. Schouery
rafael@ic.unicamp.br

¹ Institute of Computing, University of Campinas, Campinas, SP, Brazil

² Institute of Mathematics and Technology, Federal University of Catalão, Catalão-GO, Brazil

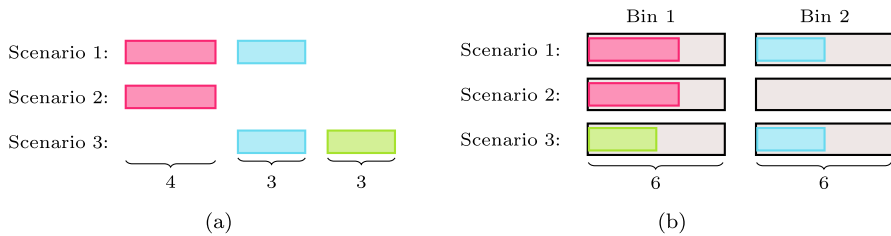


Fig. 1 An example of the BPPS. In Fig. 1a, there is a simple instance of the BPPS with bin capacity 6, an item of size 4 in scenarios 1 and 2, an item of size 3 in scenarios 1 and 3, and an item of size 3 in scenario 3. An optimal solution for this instance is shown in Fig. 1b, using 2 bins in scenarios 1 and 3 and one bin in scenario 2

One of the current challenges to solving practical logistic problems is to deal with the uncertainty arising from real-world applications (Saint-Guillain et al. 2021; Xu et al. 2016; Juan et al. 2018; Baghalian et al. 2013). In particular, one popular way to deal with this issue is by describing scenarios. A scenario is defined as a possible outcome that may arise depending on the problem's uncertain variables. Some authors use scenarios to consider the problem description as combinatorial rather than stochastic, as, e.g., Feuerstein et al. (2014). Considering the scarcity of works that apply this concept of scenarios to deal with uncertainty in bin packing problems and the existence of practical applications, Bódis and Balogh (2019) introduced the *Bin Packing Problem with Scenarios* (BPPS).

The BPPS is a generalization of the BPP in which we are given in advance a set of scenarios, of which only one will be realized. Each item belongs to a subset of scenarios, and a packing must respect the capacity constraints of the bins for each scenario individually. That is, a packing into a single bin is feasible if, for each scenario, the sum of the sizes of each item belonging to that scenario in the bin is at most the bin capacity. Although Bódis and Balogh (2019) introduced three objective functions for the BPPS, we are concerned with the objective of minimizing the number of bins of the worst-case scenario, which is the most challenging one. Notice that if a bin has no items from a specific scenario, then that bin does not count in the solution for that scenario. In this way, the BPP is the particular case of the BPPS where there is a single scenario.

To illustrate the problem, consider the example of Fig. 1 which shows a simple instance of the BPPS with three items, three scenarios and bin capacity 6. An optimal solution for this instance is shown in Fig. 1b, using two bins in the worst scenario.

This paper is organized as follows. In Sect. 2, we present a short literature review with relevant references. In Sect. 3, we provide formal definitions to be used to contextualize the contributions of the paper. The following sections detail the main contributions of this work. In Sect. 4, we show that an approximate solution for BPPS can be obtained by solving an instance of the VBPP. In Sect. 5, we present an *asymptotic polynomial time approximation scheme* (APTAS) for the case where the number of scenarios is not a part of the input. As for practical results, Sect. 6 describes a *Variable Neighborhood Search* (VNS) algorithm for the BPPS, and Sect. 7 presents a branch-and-price algorithm to solve an exponential set-cover *Integer Linear Programming*

(ILP) model for the BPPS. The evaluation of the proposed VNS and branch-and-price algorithms is presented in Sect. 8, from experiments based on the solution of randomly generated instances. Finally, Sect. 9 presents the conclusions and directions for future research.

2 Literature Review

In this section, we present a brief literature review on algorithmic results for the BPPS and some related problems.

The BPP and its variants have been extensively investigated since the thirties (Kantorovich 1960). Such problems are among the most studied in approximation contexts. A recent survey by Coffman et al. (2013) presented over 180 references related to approximation results for the BPP and its variants. In particular, the BPP is shown to be APX-hard, and indeed no algorithm has an approximation factor smaller than $3/2$, unless $P = NP$ (Vazirani 2001). Regarding practical techniques for the BPP, a survey by Delorme et al. (2016) reviewed models and solution methods, and experimentally compared the available software tools to solve the problem. Recent practical contributions for the BPP include the generalized arc-flow formulation by Brandão and Pedroso (2016), the cooperative parallel genetic algorithm by Kucukyilmaz and Kiziloz (2018), the reflect formulation by Delorme and Iori (2019), the branch-and-cut-and-price algorithm by Wei et al. (2020b) and the application of the network flow framework of de Lima et al. (2023).

Several variants of the BPP have also been extensively investigated in the literature. For example, in the variant with fragile objects, the capacity of a bin is directly influenced by its most fragile item (Claudiaux et al. 2014). In the BPP with precedence constraints, a precedent item cannot be packed in a bin later than its successors (Pereira 2016). The variant with overlapping items assumes that some subsets of items, when packed in the same bin, occupy less capacity than the sum of their individual size (Grange et al. 2018). In the BPP with item fragmentation, items can be split and fragmentally packed (Bertazzi et al. 2019). In the generalized BPP, bins may have different cost and capacity and items are associated to a profit and are divided by compulsory (i.e., mandatory to load) and non-compulsory. The aim is to minimize the overall cost based on the bins cost and items profit (Baldi et al. 2019). In the online BPP, items arrive sequentially and each must be packed before the arrival of the next one, with no information about the next items (Balogh et al. 2019). The temporal variant considers one additional dimension in the BPP, and the bin capacity should not be violated at any unit of a discretized time horizon (Dell'Amico et al. 2020).

A generalization of the BPP that is similar to the BPPS is the *Vector Bin Packing Problem* (VBPP), where bins and items have multiple dimensions and the bin capacity must be respected in all of its dimensions. The objective is the same as in the BPP, to minimize the number of bins. Since the VBPP is a generalization of the BPP, it is also APX-hard. Woeginger (1997) showed that there is no APTAS for the two-dimensional version of this problem unless $P = NP$. The proof had a slight oversight which has been corrected recently by Ray (2024), who also showed that it is NP-hard to get an asymptotic approximation ratio better than $\frac{600}{599}$. Chekuri and Khanna (2004) gave

an asymptotic $O(\ln d)$ -approximation for the case where d is a fixed constant. Later on, this result was improved to $1 + \ln d$ by Bansal et al. (2009). Also, as noted by Christensen et al. (2017), the APTAS of Fernandez de la Vega and Lueker (1981) implies a $(d + \varepsilon)$ -approximation for the VBPP and First Fit is a $(d + \frac{7}{10})$ -competitive online algorithm for VBPP, while a $\Omega(d^{1-\varepsilon})$ lower bound is known (Azar et al. 2013). Caprara and Toth (2001) presented methods to obtain lower bounds, heuristics, and a branch-and-bound (B&B) algorithm for the VBPP with two dimensions. Alves et al. (2014) investigated dual feasible functions for this problem, which we discuss later. Buljubašić and Vasquez (2016) proposed a local search algorithms where a Tabu search and descent search with add and drop moves were used to explore the search space. In Hu et al. (2017), a set-cover model is solved by a branch-and-price (B&P) algorithm. In Heßler et al. (2018), there is a stabilized branch-and-price algorithm with dual optimal cuts. Recently, Wei et al. (2020a) developed a branch-and-price algorithm for the vector packing with two dimensions, where a goal cut approach is used to obtain lower bounds and a dynamic programming with branch-and-bound solves the pricing problem, improving previous results of the literature.

To the best of our knowledge, the work by Bódís and Balogh (2019) is the only one in the literature concerned with the BPPS. They study the problem in the online case, and give (asymptotic) competitive ratios for various algorithms. In particular, they show that Next Fit is a tight $2d$ -competitive algorithm, while algorithms in the Any Fit class are no better than d -competitive. Moreover, they give an adaptation of the Harmonic algorithm by Lee and Lee (1985) which is $1.691\dots$ -competitive.

Motivated by its interesting theoretical aspects, its general applicability and its relation to well-studied problems, this paper proposes theoretical and practical results for the BPPS.

3 Formal Definitions

In this section, we provide some definitions to contextualize the contributions of this work. For simplicity, throughout the paper, we denote, for any natural n , the set $\{1, \dots, n\}$ simply by $[n]$.

Bin Packing Problem with Scenarios (BPPS)

In the BPPS, we are given a number d of scenarios, a set $\mathcal{I} = [n]$ of n items, with each $i \in \mathcal{I}$ having a size $s_i \in \mathbb{Q}_+$ and a set of scenarios $\mathcal{K}_i \subseteq [d]$, and an unlimited number of identical bins of capacity $W \in \mathbb{Q}_+$. For each $k \in [d]$, the set of items in scenario k is denoted by $S_k = \{i \in \mathcal{I} \mid k \in \mathcal{K}_i\}$. A solution $\mathcal{B}$ of the BPPS is a partition of $\mathcal{I}$ such that for each part $B \in \mathcal{B}$ and for each scenario $k \in [d]$, $\sum_{i \in B \cap S_k} s_i \leq W$. The objective of the BPPS is to find a solution that minimizes

$$V_{BPPS}(\mathcal{B}) = \max_{k \in [d]} |\{B \in \mathcal{B} : B \cap S_k \neq \emptyset\}|.$$

The objective corresponds to minimize the number of bins of the worst-case scenario, i.e., the scenario with the largest number of bins. A closely related problem is the Vector Bin Packing Problem.

Vector Bin Packing Problem (VBPP)

In the VBPP, we are given a number d of resources, a set $\mathcal{I} = [n]$ of n items, each $i \in \mathcal{I}$ having a vector $s_i \in \mathbb{Q}_+^d$ of resource consumption, and an unlimited number of identical bins of resource capacity given as a d -dimensional vector $W = (w_1, w_2, \dots, w_d) \in \mathbb{Q}_+^d$. A solution $\mathcal{B}$ is a partition of $\mathcal{I}$ such that for each part $B \in \mathcal{B}$ and each resource $k \in [d]$, $\sum_{i \in B} s_{ik} \leq w_k$. The objective is to find a solution that minimizes the number of parts, that is

$$V_{VBPP}(\mathcal{B}) = |\mathcal{B}|.$$

For the theoretical results of this paper in Sects. 4 and 5, we assume, without loss of generality, that the instances of the BPPS and the VBPP are normalized by a proper scaling of the items and, thus, the capacity of the bins is 1 (in every dimension, when considering VBPP).

An important concept in both theoretical and practical results for the BPP and related variants is the one of *cutting pattern*: a feasible combination of items in a single bin. For the BPPS, we represent a cutting pattern p as a vector of binary coefficients indicating which items and which scenarios are present in it. We denote by $\mathcal{P}$ the set of all feasible patterns for the BPPS.

4 Approximation Algorithms from Vector Bin Packing

In this section, we show how an approximation algorithm for the BPPS can be derived through a reduction to the VBPP.

In the following, we consider a mapping between the set of instances of the BPPS and a subset of instances of the VBPP. Starting with a normalized instance $I = (d, \mathcal{I}, \mathcal{K}, s)$ of the BPPS, we construct a normalized instance $I' = (d, \mathcal{I}, s')$ of the VBPP. For that, let, for each $i \in \mathcal{I}$ and $k \in [d]$,

$$s_{ik} = \begin{cases} s_i & \text{if } i \in S_k, \\ 0 & \text{otherwise.} \end{cases}$$

It is simple to verify that a solution $\mathcal{B}$ for I is a solution for I' , and vice-versa.

Definition 1 A solution $\mathcal{B}$ is minimal for a normalized instance of the BPPS if, for any two bins $A, B \in \mathcal{B}$, there exists a scenario k , such that

$$\sum_{i \in A \cap S_k} s_i + \sum_{i \in B \cap S_k} s_i > 1.$$

Notice that, for any solution $\mathcal{B}$ for the BPPS, the number of non empty bins in any scenario is at most $|\mathcal{B}|$, therefore $V_{BPPS}(\mathcal{B}) \leq |\mathcal{B}| \leq V_{VBPP}(\mathcal{B})$. In the opposite direction, we have Theorem 1.

Theorem 1 If $\mathcal{B}$ is a minimal solution for the BPPS, then $V_{VBPP}(\mathcal{B}) \leq \sqrt{d} V_{BPPS}(\mathcal{B})$.

Proof Let $\mathcal{B}$ be a minimal feasible solution for the BPPS. We say that two bins A and B are incompatible in a scenario k if both bins, A and B , are non-empty in scenario k . Let G be a multigraph where each vertex corresponds to a bin, and for each pair of bins A and B and scenario k , there is one edge between A and B (labeled as k) if they are incompatible in k . Thus, G can have parallel edges between A and B when they are incompatible in more than one scenario.

For each scenario $k \in [d]$, we define $h_k = |\{B \in \mathcal{B} : B \cap S_k \neq \emptyset\}|$, i.e., h_k is the number of bins used by $\mathcal{B}$ in k . Let $h = \max_{k \in [d]} h_k$, and notice that, since, for each scenario, there is an edge for each two used bins, the number of edges in G is

$$|E(G)| = \sum_{k=1}^d \frac{h_k(h_k - 1)}{2} \leq d \frac{h(h - 1)}{2}.$$

Let r be the number of bins, i.e., $r = |\mathcal{B}|$. Since $\mathcal{B}$ is minimal, any two bins are incompatible in at least one scenario and G must have at least one edge between any two vertices, i.e., $|E(G)| \geq \frac{r(r-1)}{2}$. Thus,

$$\frac{r(r-1)}{2} \leq d \frac{h(h-1)}{2},$$

and, since $r \leq dh$, as each bin is non-empty in at least one scenario, we have that

$$r^2 \leq dh^2 + r - dh \leq dh^2.$$

The result follows as $r = V_{VBPP}(\mathcal{B})$ and $h = V_{BPPS}(\mathcal{B})$. $\square$

We say that an asymptotic α -approximation algorithm has constant C if, for any instance I of the problem, the algorithm finds a solution of value at most $\alpha \text{OPT}(I) + C$, where $\text{OPT}(I)$ is the value of an optimal solution for I . Note that, if C is zero, then the algorithm is an absolute α -approximation algorithm.

Corollary 1 *Suppose there exists an asymptotic α -approximation for the VBPP with constant C . Then there exists an asymptotic $\alpha\sqrt{d}$ -approximation with constant C for the BPPS.*

Proof Consider an instance I of BPPS and the instance I' of VBPP obtained as described above. Also, let $\mathcal{B}$ be the solution found by the asymptotic α -approximation with constant C for instance I' , $\mathcal{B}_V^*$ be an optimal solution for VBPP with instance I' and $\mathcal{B}_S^*$ be a minimal optimal solution for BPPS with instance I . Then, we have

$$V_{BPPS}(\mathcal{B}) \leq V_{VBPP}(\mathcal{B}) \tag{1}$$

$$\leq \alpha V_{VBPP}(\mathcal{B}_V^*) + C \tag{2}$$

$$\leq \alpha V_{VBPP}(\mathcal{B}_S^*) + C \tag{3}$$

$$\leq \alpha\sqrt{d} V_{BPPS}(\mathcal{B}_S^*) + C, \tag{4}$$

where inequality (1) follows from our previous observation that value of $\mathcal{B}$ in BPPS cannot be large that its value in VBPP, inequality (2) follows from the fact that $\mathcal{B}$ is an asymptotic α -approximation with constant C for the VBPP, inequality (3) follows from the fact that $\mathcal{B}_S^*$ is a feasible solution for the VBPP and inequality (4) follows from Theorem 1 as $\mathcal{B}_S^*$ is minimal. $\square$

The bound given by this strategy cannot improve the dependency on d asymptotically, by the following lemma.

Theorem 2 *For each $d \geq 1$, there exists an instance I of the BPPS with d scenarios and a solution $\mathcal{B}$ for I , which is minimal for the BPPS, such that $V_{VBPP}(\mathcal{B}) \geq \frac{\sqrt{d}}{2} V_{BPPS}(\mathcal{B})$.*

Proof Let $r = \lceil \sqrt{d} \rceil$.

We construct an instance $I = (d, \mathcal{I}, \mathcal{K}, s)$ that contains r items.

We consider a complete graph G with r vertices. For each vertex i of G , create an item i with size $s_i = 1$, and for each edge $\{i, j\}$ of G , create a scenario k with items $S_k = \{i, j\}$. Notice that each item is in exactly $r - 1$ scenarios, and that the number of scenarios is

$$|E(G)| = \frac{r(r-1)}{2} = \frac{\lceil \sqrt{d} \rceil (\lceil \sqrt{d} \rceil - 1)}{2} \leq \frac{(\sqrt{d} + 1)\sqrt{d}}{2} \leq d.$$

Now, we create a solution $\mathcal{B}$ with r bins, such that, for each item i , there is one bin $B_i = \{i\}$.

We claim that $\mathcal{B}$ is minimal for the BPPS. Indeed, for any two bins B_i and B_j , items i and j are contained in a common scenario k , with $S_k = \{i, j\}$, and $s_i + s_j > 1$.

Notice that $V_{VBPP}(\mathcal{B}) = r$. Let $h = V_{BPPS}(\mathcal{B})$, so h is the maximum number of non-empty bins in a given scenario. Since each scenario contains exactly 2 items, then $h = 2$. Therefore,

$$r = \lceil \sqrt{d} \rceil \geq \frac{\sqrt{d}}{2} 2 = \frac{\sqrt{d}}{2} h.$$

$\square$

Using the asymptotic $(1 + \ln d)$ -approximation algorithm of Bansal et al. (2009) for the VBPP (for constant d) and the fact that the APTAS of Fernandez de la Vega and Lueker (1981) implies a $(d + \varepsilon)$ -approximation for the VBPP (Christensen et al. 2017), we have the following result.

Corollary 2 *There exists a $(d + \varepsilon)\sqrt{d}$ -approximation for the BPPS. Also, for any constant d , there exists an asymptotic $(1 + \ln d)\sqrt{d}$ -approximation for the BPPS.*

For online algorithms, First Fit has a competitive ratio of $d + \frac{7}{10}$ for VBPP (Christensen et al. 2017). Since First Fit produces a minimal solution, we have the following result.

Corollary 3 *First Fit is $(d + \frac{7}{10})\sqrt{d}$ -competitive algorithm for the BPPS.*

For example, for $d = 2$ and $d = 3$, this corollary implies online algorithms with competitive ratios $2.7\sqrt{2} \approx 3.82$ and $3.7\sqrt{3} \approx 6.41$, respectively.

5 An Asymptotic Polynomial Time Approximation Scheme

In this section, we present an APTAS for the BPPS when the number of scenarios d is constant, that is, when d is not a part of the input.

First, we observe that, if all items are large and the number of distinct item sizes is bounded by a constant, then the number of valid patterns of items in a bin is also bounded by a constant. It implies that for this restricted case, a polynomial algorithm can be readily obtained by simply enumerating the frequencies of all patterns.

To obtain such a restricted instance, we use the *linear grouping* technique, following Fernandez de la Vega and Lueker (1981), by grouping items of similar sizes and creating a map from smaller to larger items. A naive approach does not work, however, since one must take into account the scenarios. Thus, we group items according to their scenarios separately, but we do the mapping simultaneously.

Finally, to solve a general instance, we combine small items into artificial large items. Again, this is done on a per-scenario basis, so as only compatible items are combined. If the small items do not fit exactly into the area of the artificial items, then this change may cause bins to be slightly augmented. The surpassing items are then relocated to new bins greedily, increasing the number of bins by just a fraction of the optimal value.

In the following, we use the notion of type. The *type* of item i is the set of scenarios which contain i . The set of all types is denoted by $\mathcal{T}$. Also, the value of an optimal solution for an instance I is denoted by $\text{OPT}(I)$.

5.1 Restricted instances

In this subsection, we consider instances of the BPPS in which the items are large and the number of distinct sizes is bounded by a constant. Precisely, let V be a set of positive numbers and consider a constant ε , with $0 < \varepsilon \leq 1/4$ and $1/\varepsilon$ is an integer. A (normalized) instance $(d, \mathcal{I}, \mathcal{K}, s)$ of the BPPS is said to be *V-restricted* if $\{s_i : i \in \mathcal{I}\} = V$ and $s_i \geq \varepsilon^2$ for every $i \in \mathcal{I}$.

Recall that a bin $B \subseteq \mathcal{I}$ is feasible if, for any scenario $k \in [d]$, $\sum_{i \in B \cap S_k} s_i \leq 1$. For a type $t \in \mathcal{T}$ and $v \in V$, let B_{tv} be the subset of items in B with type t and size v . Notice that $|B_{tv}|$ is at most $1/v \leq 1/\varepsilon^2$, as the items B_{tv} have size at least ε^2 and all of them are contained in a single bin for any scenario of t . Notice also that the family of all sets B_{tv} form a partition of B . The *pattern* of B is the vector $(|B_{tv}|)_{t \in \mathcal{T}, v \in V}$. As there is a constant number of types, if $|V|$ is constant, then we can bound the number of patterns by enumerating the possible values of $|B_{tv}|$, which is shown in Lemma 1. This lemma is used later on to enumerate all possible solutions for a V -restricted instance in polynomial time (when $|V|$ is constant).

Lemma 1 *If $|V|$ is bounded by a constant, the number of distinct patterns for a V -restricted instance is bounded by a constant.*

Proof Consider a feasible bin B . Since $s_i \geq \varepsilon^2$, for any $i \in B$,

$$|B|\varepsilon^2 \leq \sum_{i \in B} s_i \leq \sum_{k=1}^d \sum_{i \in B \cap S_k} s_i \leq \sum_{k=1}^d 1 = d,$$

where the last inequality follows because B is feasible. It means that the number of items in any bin is at most d/ε^2 . Since a vector corresponding to a pattern has $|\mathcal{T}||V|$ elements, the number of patterns is at most $(1 + d/\varepsilon^2)^{|\mathcal{T}||V|}$, which is constant since d is constant. $\square$

Lemma 2 *If $|V|$ is constant, then an optimal solution of a V -restricted instance can be computed in polynomial time.*

Proof Consider a V -restricted instance I , and let M be the set of distinct patterns for I . Given a solution $\mathcal{B}$ and a pattern $p \in M$, the number of bins in $\mathcal{B}$, which has (a non-empty) pattern p , is denoted by n_p . Clearly $n_p \leq |\mathcal{I}|$ for every $p \in M$, as there are only $|\mathcal{I}|$ items, and each bin has at least one item. The configuration of $\mathcal{B}$ is the vector $(n_p)_{p \in M}$. Thus, the number of possible configurations is bounded by $|\mathcal{I}|^{|M|}$, which is polynomial since, by Lemma 1, $|M|$ is constant.

Notice that, given a vector $(n_p)_{p \in M}$, one can either find a solution $\mathcal{B}$ with this configuration, or decide there is no such solution. Indeed, it is sufficient to count the total number of items of each type and size over all the patterns. Therefore, one can list all configurations in polynomial time and return the one with the minimum value. $\square$

Notice that Lemma 1 is true only because we are considering d as a constant. If d is part of the input, then, as $\mathcal{T}$ can have up to 2^d elements, the number of patterns can be exponential in d . In this case, we would not have a polynomial time algorithm as in Lemma 2. Thus, considering d as a constant is crucial for the running time of our algorithm.

5.2 An APTAS for large items

Suppose now that we have an instance $I = (d, \mathcal{I}, \mathcal{K}, s)$, such that all items are large, i.e., for each $i \in \mathcal{I}$, we have $s_i \geq \varepsilon^2$, but the number of distinct sizes is not necessarily bounded by a constant.

Definition 2 Given instances $I = (d, \mathcal{I}, \mathcal{K}, s)$ and $\bar{I} = (d, \bar{\mathcal{I}}, \bar{\mathcal{K}}, \bar{s})$, we say that I dominates $\bar{I}$ if there exists a subset $\mathcal{I}' \subseteq \mathcal{I}$ and a bijection $f : \mathcal{I}' \rightarrow \bar{\mathcal{I}}$, such that, for every $i \in \mathcal{I}'$, items i and $f(i)$ have the same type and $s_i \geq \bar{s}_{f(i)}$. In this case, we write $I \succcurlyeq \bar{I}$.

Lemma 3 *If $I \succcurlyeq \bar{I}$, then $OPT(I) \geq OPT(\bar{I})$.*

$\square$

In the following, we create an instance $\bar{I} = (d, \bar{\mathcal{I}}, \bar{\mathcal{K}}, \bar{s})$ with $I \succcurlyeq \bar{I}$, such that all items of $\bar{I}$ are large and the number of distinct sizes is bounded by a constant.

First, for each $t \in \mathcal{T}$, let $\mathcal{I}_t$ be the set of items of type t and let $m = \frac{2^d}{\varepsilon^3} - 1$. We consider the set $\mathcal{I}_t$ sorted in non increasing order according to the items' sizes, and build $m + 1$ groups of consecutive items, obtaining a partition $\{G_t^0, G_t^1, \dots, G_t^m\}$, where the first m groups have size $\lceil |\mathcal{I}_t|/(m + 1) \rceil$, and the last group either has the same size, or is smaller.

Now, for each $t \in \mathcal{T}$ and $\ell \in \{1, 2, \dots, m\}$, we create a set $\tilde{G}_t^\ell$ as follows: for each item $i \in G_t^\ell$, add an item to $\tilde{G}_t^\ell$ with the same set of scenarios t , and with size equal to the size of the largest item in G_t^ℓ . Let $\tilde{\mathcal{I}}$ be the union of all sets $\tilde{G}_t^\ell$, and $\bar{I}$ be the instance induced by $\tilde{\mathcal{I}}$.

Lemma 4 $I \succcurlyeq \bar{I}$.

Proof For $\ell \in \{1, 2, \dots, m\}$, as $|G_t^{\ell-1}| \geq |G_t^\ell| = |\tilde{G}_t^\ell|$, we can find a bijection f between a subset of $G_t^{\ell-1}$ and $\tilde{G}_t^\ell$ for $\ell \in \{1, 2, \dots, m\}$ such that i and $f(i)$ has the same type. Finally, as $s_i \geq s_j$ for $i \in G_t^{\ell-1}$ and $j \in G_t^\ell$ and $\bar{s}_{f(i)}$ is the maximum size of an item in G_t^ℓ , we have that $s_i \geq \bar{s}_{f(i)}$. $\square$

For a type $t \in \mathcal{T}$, the group of the largest items is G_t^0 . For each item i in this group, we pack it into a separate bin $\{i\}$. By joining all these bins, we obtain a packing $\mathcal{P}^0$.

Lemma 5 $V_{BPPS}(\mathcal{P}^0) \leq \varepsilon \text{OPT}(I) + 2^d$.

Proof Let $t \in \mathcal{T}$ and fix an arbitrary scenario k of t . We consider an optimal solution for I , and let $\mathcal{P}_k$ be the set of bins used in scenario k in this solution. Therefore, $|\mathcal{P}_k| \leq \text{OPT}(I)$. Recall that $\mathcal{I}_t$ is the set of items of type t . Observe that, since k belongs to t , the set of items included in bins of $\mathcal{P}_k$ must contain $\mathcal{I}_t$. As, for each $i \in \mathcal{I}_t$, $s_i \geq \varepsilon^2$,

$$\varepsilon^2 |\mathcal{I}_t| \leq \sum_{i \in \mathcal{I}_t} s_i \leq \sum_{B \in \mathcal{P}_k} \sum_{i \in B \cap \mathcal{I}_t} s_i \leq \sum_{B \in \mathcal{P}_k} 1 = |\mathcal{P}_k| \leq \text{OPT}(I).$$

This implies that the value of $\mathcal{P}^0$ can be bounded by

$$\sum_{t \in \mathcal{T}} |G_t^0| = \sum_{t \in \mathcal{T}} \lceil \frac{\varepsilon^3}{2^d} |\mathcal{I}_t| \rceil \leq 2^d + \sum_{t \in \mathcal{T}} \frac{\varepsilon \text{OPT}(I)}{2^d} \leq \varepsilon \text{OPT}(I) + 2^d.$$

To obtain a packing of the remaining items, we solve instance $\bar{I}$. For this, let V be the set of distinct sizes in $\bar{I}$. For each $t \in \mathcal{T}$, we created m groups, where each group has items of the same size. Therefore, $|V| \leq 2^d m$. This means that $\bar{I}$ is a V -restricted instance for $|V|$ bounded by a constant, since d and m are constant. Using Lemma 2, we obtain a solution $\bar{\mathcal{P}}^1$ for $\bar{I}$ in polynomial time. Since $I \succcurlyeq \bar{I}$, we can obtain a solution $\mathcal{P}^0 \cup \mathcal{P}^1$ for I , where $\mathcal{P}^1$ is obtained from $\bar{\mathcal{P}}^1$ by replacing items of $\tilde{G}_t^\ell$ by items of G_t^ℓ . Let I^1 be the instance obtained from the items of $\mathcal{P}^1$. Observe that $\mathcal{P}^1$ is feasible, since we replaced items of larger size which appear in the same set of scenarios, thus for each scenario, the capacity constraint of each bin remains satisfied.

Lemma 6 $V_{BPPS}(\mathcal{P}^1) \leq \text{OPT}(I)$.

Proof Note that the number of unused bins in each scenario is the same for $\mathcal{P}^1$ and $\bar{\mathcal{P}}^1$, thus $V_{BPPS}(\mathcal{P}^1) = V_{BPPS}(\bar{\mathcal{P}}^1)$. Since, $\bar{\mathcal{P}}^1$ is optimal for $\bar{I}$, we have $V_{BPPS}(\bar{\mathcal{P}}^1) = \text{OPT}(I^1)$. Now, using Lemma 3 and the fact that $I \succcurlyeq \bar{I}$, we have $\text{OPT}(I^1) \leq \text{OPT}(I)$. $\square$

Combining the previous results, one obtains an APTAS for instances with large items only.

Lemma 7 Suppose for every $i \in \mathcal{I}$, $s_i \geq \varepsilon^2$. Then one can find in polynomial time a packing $\mathcal{P}$ of $\mathcal{I}$ with $V_{BPPS}(\mathcal{P}) \leq (1 + \varepsilon) \text{OPT}(I) + 2^d$.

Proof Define $\mathcal{P} = \mathcal{P}^0 \cup \mathcal{P}^1$. Notice that $\mathcal{P}$ contains every item of $\mathcal{I}$, and thus it is feasible. Using Lemmas 5 and 6,

$$V_{BPPS}(\mathcal{P}) \leq V_{BPPS}(\mathcal{P}^0) + V_{BPPS}(\mathcal{P}^1) \leq \varepsilon \text{OPT}(I) + 2^d + \text{OPT}(I).$$

5.3 An APTAS for the general case

In the general case of the BPPS, an instance $I = (d, \mathcal{I}, \mathcal{K}, s)$ may contain large and small items. Let $\mathcal{L} = \{i \in \mathcal{I} : s_i \geq \varepsilon^2\}$ be the set of large items, and $\mathcal{S} = \mathcal{I} \setminus \mathcal{L}$ be the set of small items.

To obtain a packing of $\mathcal{L} \cup \mathcal{S}$, we first replace $\mathcal{S}$ by the set of large items, $\hat{\mathcal{S}}$, which we define as follows. For each type $t \in \mathcal{T}$, let $\mathcal{S}_t$ be the set of small items with type t . Now, let $\hat{\mathcal{S}}_t$ be a set of $\lceil \sum_{i \in \mathcal{S}_t} s_i / (\varepsilon - \varepsilon^2) \rceil$ items, such that each $j \in \hat{\mathcal{S}}_t$ has size $\hat{s}_j = \varepsilon$ and is of type t . The set $\hat{\mathcal{S}}$ is the union of sets $\hat{\mathcal{S}}_t$ for every type t .

Thus, we create an instance $\hat{I} = (d, \hat{\mathcal{I}}, \hat{\mathcal{K}}, \hat{s})$ whose set of items is $\hat{\mathcal{I}} = \mathcal{L} \cup \hat{\mathcal{S}}$, each of which has size at least ε^2 . Lemma 8 compares the optimal value of I and $\hat{I}$.

Lemma 8 $\text{OPT}(\hat{I}) \leq (1 + 2^{d+1}(d+1)\varepsilon) \text{OPT}(I) + 1$.

Proof Consider an optimal solution $\mathcal{P}^*$ for I . For each bin $B \in \mathcal{P}^*$ and type $t \in \mathcal{T}$, we define $B_t = B \cap \mathcal{S}_t$, i.e., B_t is the set of small items with type t in bin B . Then, we replace the items in B_t by $\lceil \sum_{i \in B_t} s_i / (\varepsilon - \varepsilon^2) \rceil$ new items of size ε , and with the same scenarios of t . We call $\mathcal{P}'$ this modified packing.

Observe that $\mathcal{P}'$ may be infeasible, since new items may surpass the bins' capacity. We can obtain a feasible solution $\mathcal{P}''$ by relocating some created items from $\mathcal{P}'$ to new bins. For each type t and each bin $B \in \mathcal{P}'$, the number of items needed to be picked up for relocation is at most the additional area divided by ε rounded up, that is,

$$\begin{aligned} \left\lceil \frac{\left\lceil \sum_{i \in B_t} \frac{s_i}{\varepsilon - \varepsilon^2} \right\rceil \varepsilon - \sum_{i \in B_t} s_i}{\varepsilon} \right\rceil &\leq 1 + \frac{\varepsilon + \varepsilon \sum_{i \in B_t} \frac{s_i}{\varepsilon - \varepsilon^2} - \sum_{i \in B_t} s_i}{\varepsilon} \\ &= 2 + \frac{1}{1 - \varepsilon} \sum_{i \in B_t} s_i \\ &\leq 2 + 2 \sum_{i \in B_t} s_i, \end{aligned}$$

where the last inequality follows from the fact that $\varepsilon \leq 1/2$. Let R be the set of all picked items. Observe that $|\mathcal{P}^*| \leq d \text{OPT}(I)$, since each bin is not empty for at least one scenario. Thus, as there are 2^d types and $\sum_{B \in \mathcal{P}^*} \sum_{i \in B_t} s_i \leq \text{OPT}(I)$, the number of items in R is at most

$$\begin{aligned} \sum_{t \in \mathcal{T}} \sum_{B \in \mathcal{P}^*} \left(2 + 2 \sum_{i \in B_t} s_i \right) &\leq 2^{d+1} |\mathcal{P}^*| + 2^{d+1} \text{OPT}(I) \\ &\leq 2^{d+1} d \text{OPT}(I) + 2^{d+1} \text{OPT}(I) \\ &= 2^{d+1} (d + 1) \text{OPT}(I). \end{aligned}$$

Since $1/\varepsilon$ is an integer, one can rearrange all items of R into at most

$$\left\lceil \frac{2^{d+1} (d + 1) \text{OPT}(I)}{1/\varepsilon} \right\rceil$$

new bins of unit capacity. Let $\mathcal{R}$ be this set of new bins, and obtain a feasible packing $\mathcal{P}''$ of I , by making a copy of $\mathcal{P}'$, removing the items of R , and adding the bins of $\mathcal{R}$.

Notice that $V_{BPPS}(\mathcal{P}') = V_{BPPS}(\mathcal{P}^*) = \text{OPT}(I)$. Using these facts, the value of $\mathcal{P}''$ can be estimated as

$$\begin{aligned} V_{BPPS}(\mathcal{P}'') &\leq V_{BPPS}(\mathcal{P}') + |\mathcal{R}| \\ &\leq \text{OPT}(I) + \left\lceil \frac{2^{d+1} (d + 1) \text{OPT}(I)}{1/\varepsilon} \right\rceil \\ &\leq \text{OPT}(I) + 2^{d+1} (d + 1) \varepsilon \text{OPT}(I) + 1. \end{aligned}$$

We notice that $\mathcal{P}''$ contains all large items $\mathcal{L}$ and, for each type t , a certain number of items with scenarios t and size ε . Thus, to obtain a packing for $\hat{I}$ from $\mathcal{P}''$, it is sufficient to show that for each t , the number of created items corresponding to t is not smaller than $|\hat{\mathcal{S}}_t|$. Indeed, for a given t , the number of created items is

$$\sum_{B \in \mathcal{P}^*} \left\lceil \sum_{i \in B_t} s_i / (\varepsilon - \varepsilon^2) \right\rceil \geq \left\lceil \sum_{i \in \mathcal{S}_t} s_i / (\varepsilon - \varepsilon^2) \right\rceil = |\hat{\mathcal{S}}_t|.$$

To conclude the proof, we create a packing $\hat{\mathcal{P}}$ for $\hat{I}$ by removing the exceeding items of $\mathcal{P}''$. Since removing such items can only decrease the objective value, $V_{BPPS}(\hat{\mathcal{P}}) \leq V_{BPPS}(\mathcal{P}'')$. $\square$

Finally, to obtain a packing for I , one can first obtain a packing of $\hat{I}$, then replace items from $\hat{\mathcal{S}}_t$ by items in $\mathcal{S}_t$. This leads to Theorem 3.

Theorem 3 For every constant $\varepsilon' > 0$, one can find in polynomial time a packing $\mathcal{P}$ of $\mathcal{I}$ such that $V_{BPPS}(\mathcal{P}) \leq (1 + \varepsilon') \text{OPT}(I) + 2 + 2^d$.

Proof Define $r = 2^{d+1}(d + 1)$ and define ε as the largest number such that $\varepsilon \leq \min(\frac{\varepsilon'}{3r}, 1/4)$ and $1/\varepsilon$ is an integer. Let $\hat{I}$ be the instance obtained from I by replacing, for each type, the items smaller than ε^2 with items of size ε , as previously mentioned. Since each item of $\hat{I}$ has size at least ε^2 , by Lemma 7, we obtain a packing $\hat{\mathcal{P}}$ for $\hat{I}$ with value $V_{BPPS}(\hat{\mathcal{P}}) \leq (1 + \varepsilon) \text{OPT}(\hat{I}) + 2^d$ in polynomial time.

For each $t \in \mathcal{T}$, we place all items of $\mathcal{S}_t$ over the items of $\hat{\mathcal{S}}_t$. Precisely, we do the following: start by making a copy $\mathcal{P}$ of $\hat{\mathcal{P}}$; select a set A of unpacked items in $\mathcal{S}_t$ whose total size is at least $\varepsilon - \varepsilon^2$ and at most ε . If there is not such a set A , then let A be the remaining set of unpacked items in $\mathcal{S}_t$. Find a bin $B \in \mathcal{P}$ with an item $j \in \hat{\mathcal{S}}_t \cap B$ and replace j by the items of A . Since there are $\lceil \sum_{i \in \mathcal{S}_t} s_i / (\varepsilon - \varepsilon^2) \rceil$ items in $\hat{\mathcal{S}}_t$, and each group A , except perhaps one of them, has size at least $\varepsilon - \varepsilon^2$, we always find an unused item $j \in \hat{\mathcal{S}}_t$; repeat these steps while there are unpacked items in $\mathcal{S}_t$ for some t .

The resulting packing $\mathcal{P}$ contains all items in $\mathcal{I}$. As no bin was added, $\mathcal{P}$ has the same value of $\hat{\mathcal{P}}$. Using Lemma 8, we finally obtain

$$\begin{aligned} V_{BPPS}(\mathcal{P}) &= V_{BPPS}(\hat{\mathcal{P}}) \leq (1 + \varepsilon) \text{OPT}(\hat{I}) + 2^d \\ &\leq (1 + \varepsilon)[(1 + 2^{d+1}(d + 1)\varepsilon) \text{OPT}(I) + 1] + 2^d \\ &= (1 + r\varepsilon^2 + r\varepsilon + \varepsilon) \text{OPT}(I) + 2^d + 1 + \varepsilon \\ &\leq (1 + 3r\varepsilon) \text{OPT}(I) + 2^d + 1 + \varepsilon \\ &\leq (1 + \varepsilon') \text{OPT}(I) + 2 + 2^d. \end{aligned}$$

6 A Variable Neighborhood Search Algorithm

Variable Neighborhood Search (VNS) is a metaheuristic that searches different neighborhood structures with systematic changes of neighborhood to find better solutions and escape from local optima. It was initially proposed by Mladenović and Hansen (1997) and has applications on several optimization problems, including cutting and packing (see, e.g. Puchinger and Raidl (2008); Fleszar and Hindi (2002); Beltrán et al. (2004)). In this section, we provide an overview of the VNS algorithm used in this work, as well as discuss its key problem-specific characteristics in the context of the BPPS.

6.1 Overview of the algorithm

In the VNS framework, an initial solution is taken as input, then iteratively improved through a diversification step followed by an intensification step. Diversification aims at exploring different parts of the solution space, while intensification aims at improving the current solution as much as possible.

Diversification is done through a SHAKE procedure that, given a solution x and a neighborhood structure κ , simply produces a random solution in the κ -th neighborhood of x .

Intensification is done through a local search procedure that explores multiple neighborhood structures. This procedure implements the variable neighborhood descent approach which, given a solution x and the number of neighborhood structures considered N_k , produces a solution that is local optima in all neighborhood structures. This is achieved by looking for the best solution in a specific neighborhood structure: if an improving solution is found, the search is restarted from the first neighborhood structure; if no improving solution is found, the search proceeds to the next neighborhood structure. This process is repeated until no improving solution is found at the last neighborhood structure. Algorithm LOCAL- SEARCH details this procedure.

LOCAL- SEARCH($x, N_{\max}$)

```

1   $\kappa = 1$ 
2  repeat
3       $x' = \arg \min_{y \in N_{\kappa}(x)} f(y)$  // Find the first best neighbor of  $x$ 
4      if  $f(x') < f(x)$  // There has been an improvement
5           $x = x'$ 
6           $\kappa = 1$ 
7      else // Local optima, change neighborhood
8           $\kappa = \kappa + 1$ 
9  until  $\kappa == N_{\max}$ 
```

The VNS algorithm takes as an input an initial solution x , a number of neighborhood structures $N_{\max}$, a time limit $t_{\max}$ and a limit $c_{\max}$ on the number of iterations. At each iteration, a random solution is selected from the current neighborhood through the SHAKE procedure, and then it is improved with the LOCAL- SEARCH algorithm. The search advances to the next neighborhood whenever the local search fails to find an improving solution. The main loop can be interrupted whenever either $t_{\max}$ seconds or $c_{\max}$ iterations without improving the solution have passed. In the experiments of Sect. 8, we use $t_{\max} = 1800$ and $c_{\max} = 500$, which were obtained through preliminary testing. Algorithm VNS is described in detail as follows.

VNS($x, N_{\max}, t_{\max}, c_{\max}$)

```

1   $t = 0, c = 0$ 
2  while  $t < t_{\max}$  and  $c < c_{\max}$ 
3       $\kappa = 1$ 
4       $improvement = \text{False}$  // Assume there will be no improvement
5      repeat
6           $x' = \text{SHAKE}(x, \kappa)$  // Get a random solution in  $N_{\kappa}(x)$ 
7           $x'' = \text{LOCAL-SEARCH}(x', N_{\max})$  // Perform VND to improve  $x'$ 
8          if  $f(x'') < f(x)$  // There has been an improvement
9               $x = x''$ 
10              $\kappa = 1$ 
11              $improvement = \text{True}$ 
12         else // Local optima, change neighborhood
13              $\kappa = \kappa + 1$ 
14     until  $\kappa == N_{\max}$ 
15     if  $improvement$  // There has been an improvement
16          $c = 0$ 
17     else // No improvement in this iteration
18          $c = c + 1$ 
19      $t = \text{CPU-TIME}()$  // Update processing time.
```

6.2 Initial solution

The VNS algorithm requires an initial feasible solution as an input. For this purpose, we implement a simple adaptation of the *first-fit decreasing* (FFD) heuristic for the BPP (see, e.g. Coffman et al. (2013)). This heuristic works as follows: items are sorted in non-increasing order of their sizes; each item is then packed in the first open bin where it fits, considering the capacity constraints in each scenario; if no such bin is found, then a new one is created.

6.3 Neighborhood structures

A neighborhood structure specifies a well-defined way to move from any given solution into another solution that is “close” to the first one in the solution space. We refer to these neighborhood structures as a “movement” that takes one solution as input and, depending on structure-dependent input parameters, returns a neighbor solution. In this work, we consider only movements that lead to feasible solutions, so each different structure defines their own feasible solution space. Let b be the number of bins in the current solution. We define four neighborhood structures for the BPPS in the following order:

- N_1 (swap): Given items i and j packed into different bins, move i to the bin currently containing j , then move j to the bin that contained i . The size of this neighborhood is $O(n^2)$;

- N_2 (move): Given an item i and a bin B that currently does not contain i , move i to B . The size of this neighborhood is $O(nb)$;
- N_3 (move-2): Given items i and j packed into bin B_1 and bins B_2 and B_3 , move these items to B_2 and B_3 , respectively. The size of this neighborhood is $O(n^2b^2)$;
- N_4 (repack): Given a bin B , remove its items from the solution and repack them in the first bin that can accommodate each of them, respecting the order that they appeared in B . If an item cannot be accommodated in any other bin, it will be repacked in B . The size of this neighborhood is $O(b)$.

Notice that, different from the BPP, in the BPPS it is possible that swapping two items (as described in N_1) may result in a decrease of objective value. All the other structures can decrease the number of bins in a solution by moving one or more items to other bins. Hence, all proposed neighborhood structures can improve the solution objective value.

The ordering of the neighborhood structures is another important design choice, since the first neighborhoods can potentially be used much more often. Because of this, neighborhoods N_1 , N_2 , are prioritized over N_3 due to their smaller size. Although the neighborhood structure N_4 is the smallest, the evaluation of single a movement in this structure can takes $O(n)$ since all items of a given bin must be repacked. Because of this, we consider N_4 as the last neighborhood.

6.4 Objective function

In order to properly rank neighboring solutions, a sensitive objective function must be used to guide the local search. Since the objective function for the BPPS is likely to have the same value for many solutions in a same neighborhood, we define a fitness function that penalizes solutions in which the bin occupation per scenario is small, in order to break ties among similar solutions.

Let $\mathcal{B}$ be a solution for the BPPS and $\mathcal{B}_k$ be the subset of bins that contain items in scenario k , where $|\mathcal{B}_k|$ is the number of bins used in scenario k . Furthermore, let $\text{ocp}(k, B)$ be the total size of the items in bin B from scenario k . Given a solution $\mathcal{B}$, the VNS algorithm accepts a neighbor solution $\mathcal{B}'$ if $f(\mathcal{B}) > f(\mathcal{B}')$, for f defined as:

$$f(\mathcal{B}) = V_{BPPS}(\mathcal{B}) - \sum_{k \in \mathcal{K}} \sum_{B \in \mathcal{B}_k} \left(\frac{\text{ocp}(k, B)}{|\mathcal{B}_k|W} \right)^2.$$

Consider the example in Fig. 1, with 3 items, 3 scenarios, and bin capacity 6. The solution shown in Fig. 1b has value $2 - ((4/12)^2 + (3/12)^2 + (4/6)^2 + (3/12)^2 + (3/12)^2) \approx 1.26$. Now consider moving the green item of size 3 from Bin 1 to Bin 2 (neighborhood structure N_2). The resulting solution has value $2 - ((4/12)^2 + (3/12)^2 + (4/6)^2 + (6/6)^2) \approx 0.38$, which despite using the same number of bins in the worst scenario, makes a better use of the space in the bins for each scenario. In this case, the algorithm would accept the new solution since it has smaller surrogate objective function value.

7 A Branch-and-Price Algorithm

We propose an exact method for the BPPS based on column generation and branch-and-bound, that is, a branch-and-price algorithm. This method consists of modelling the problem as an ILP with a very large (usually exponential) number of variables, which are generated and added to the model only as they are needed, in each node of the branch-and-bound tree. Branch-and-price has been used successfully to solve practical instances of the BPP and several of its variants (Delorme et al. 2016; Sadykov and Vanderbeck 2013; Dell’Amico et al. 2020; Borges et al. 2020; Bettinelli et al. 2010).

One of the most successful formulations for the classical BPP and variants is based on associating a variable to each cutting pattern and solving a set-cover model to choose which patterns to use in the solution (see, e.g., Delorme et al. (2016)). In practice, these formulations lead to models with an exponential number of variables, as the number of cutting patterns is usually exponential with respect to the number of items. Due to this, to solve practical instances with such models, one has to rely on column generation to solve the corresponding linear relaxation.

The *column generation method*, proposed by Ford and Fulkerson (1958) and generalized by Dantzig and Wolfe (1961), solves the linear relaxation of models with a large number of variables. Gilmore and Gomory (1961, 1963) were the first to present practical experiments based on column generation, by solving the linear relaxation of an exponential model for the BPP. The column generation method solves the linear relaxation by iteratively solving the *restricted master problem* (RMP), which considers only a subset of variables (columns) of the original model. At each iteration, the method has to determine the existence of non-basic columns which are candidate to improve the current solution of the RMP, i.e., columns with negative reduced cost (on minimization problems). If the method determines that no such column exists, then the solution of the RMP is an optimal solution for the original linear relaxation, providing a bound for the integer problem.

7.1 A pattern-based model

Recall that $\mathcal{P}$ is the set of all cutting patterns for the BPPS. A cutting pattern $p \in \mathcal{P}$ can be written as a vector $(a_{1p}, \dots, a_{np}, b_{1p}, \dots, b_{dp})$, such that each coefficient a_{ip} indicates whether item i is used in pattern p , and each coefficient b_{kp} indicates whether at least one item of scenario k is used in pattern p . With this, the following exponential ILP formulation models the BPPS:

$$\text{minimize } \mathcal{F}, \quad (5)$$

$$\text{s.t. : } \sum_{p \in \mathcal{P}} a_{ip} X_p \geq 1, \quad \forall i \in \mathcal{I}; \quad (6)$$

$$\sum_{p \in \mathcal{P}} b_{kp} X_p \leq \mathcal{F}, \quad \forall k \in \mathcal{K}; \quad (7)$$

$$X_p \in \{0, 1\}, \quad \forall p \in \mathcal{P}; \quad (8)$$

$$\mathcal{F} \in \mathbb{Z}_+. \quad (9)$$

For each pattern $p \in \mathcal{P}$, binary variable X_p is equal to 1 if and only if p is used in the solution. The integer variable $\mathcal{F}$ represents an upper bound on the number of bins (patterns) that is used by each individual scenario. The objective function (5) is to minimize $\mathcal{F}$. Constraints (6) guarantees that every item belongs to at least one cutting pattern of the solution. Constraints (7) ensures that, for every scenario $k \in \mathcal{K}$, the number of bins that packs at least one item from k is at most $\mathcal{F}$. The domain of variables is in (8) and (9).

Next, we describe a column generation algorithm to solve the linear relaxation of (5)–(9). As the initial set of columns for the RMP, we consider each cutting pattern with a single item. The dual solution of the RMP consists of a value $\alpha_i \geq 0$ for each item i associated with a constraint in (6), and a value $\beta_k \leq 0$ for each scenario k associated with a constraint in (7). The dual solution is used to solve the *pricing problem*, which determines a column with minimum reduced cost, that is

$$\min_{p \in \mathcal{P}} \left\{ c_p - \left(\sum_{i \in \mathcal{I}} \alpha_i a_{ip} + \sum_{k \in \mathcal{K}} \beta_k b_{kp} \right) \right\},$$

which is equivalent to

$$\max_{p \in \mathcal{P}} \left\{ \sum_{i \in \mathcal{I}} \alpha_i a_{ip} + \sum_{k \in \mathcal{K}} \beta_k b_{kp} \right\}. \quad (10)$$

7.2 The pricing problem

In order to generate a new column, we need to find a feasible pattern that optimizes (10). Since it includes a term for each item and a term for each scenario used in the pattern, we can describe the pricing problem as the following Knapsack Problem with Scenarios.

Knapsack Problem with Scenarios (KPS)

In the KPS, we are given: a number d of scenarios; a set $\mathcal{I} = [n]$ of n items, with each item $i \in \mathcal{I}$ having a size s_i , item-value $v_i \in \mathbb{Q}$, and a set of scenarios $\mathcal{K}_i \subseteq [d]$; a vector $u \in \mathbb{Q}^d$ of scenario-values, and a knapsack of capacity $W \in \mathbb{Q}_+$. For simplicity, for each item $i \in \mathcal{I}$, define:

$$s_i^k = \begin{cases} s_i, & \text{if item } i \text{ is in the scenario } k; \\ 0, & \text{otherwise.} \end{cases}$$

A solution is a subset B of $\mathcal{I}$ such that for each scenario $k \in \mathcal{K}$, $\sum_{i \in B} s_i^k \leq W$. The objective is to find a solution that maximizes the sum of the item-values and scenario-values for the selected items, that is, find B that maximizes

$$\sum_{i \in B} v_i + \sum_{k: s_i^k > 0, i \in B} u_k.$$

Theorem 4 *KPS is NP-hard in the strong sense.*

Proof Let us recall the Maximum Independent Set problem, which given a graph G consists of finding a subset of maximum cardinality of its vertices such that no two of its elements are adjacent in G . This problem is known to be strongly NP-hard (Garey and Johnson 1978), so all we need to do is present a polynomial reduction from this problem into the KPS.

Let $I = G(V, E)$ be an instance of the Maximum Independent Set problem. We construct an instance $I' = (d, \mathcal{I}, s, v, \mathcal{K}, u, W)$ of the KPS such that $|\mathcal{I}| = |V|$, $d = |V|$. For each item $i \in \mathcal{I}$ there is a scenario $k_i \in [d]$ associated with it. We define $\mathcal{K}_i = \{k_i\} \cup \{k_j : (i, j) \in E\}$ and we let $s_i^k = 1$ if $k \in \mathcal{K}_i$. We define $v_i = 1$ for all $i \in \mathcal{I}$, $u_k = 0$ for all $k \in [d]$ and $W = 1$.

Now notice that there is a bijection between solutions of I and I' which preserves the value of the objective function: the items are all unit-valued and the scenarios are zero-valued; two items corresponding to two adjacent vertices of V cannot be in the same bin of I' because they share a scenario. $\square$

As a consequence of Theorem 4, our pricing subproblem does not admit a pseudo-polynomial time algorithm, unless $P = NP$, thus, we propose the following ILP model to solve it.

We can use the dual solution (α, β) of the RMP as item-values and scenario-values (u, v) to describe the following model for the pricing subproblem

$$\text{maximize } \sum_{i \in \mathcal{I}} \alpha_i A_i + \sum_{k \in \mathcal{K}} \beta_k B_k \quad (11)$$

$$\text{s.t. : } \sum_{i \in \mathcal{I}} s_i^k A_i \leq W B_k, \quad \forall k \in \mathcal{K}; \quad (12)$$

$$A_i, B_k \in \{0, 1\}, \quad \forall i \in \mathcal{I}, k \in \mathcal{K}. \quad (13)$$

In model (11)-(13), variables A_i are related to the coefficients a_{ip} , and variables B_k are related to the coefficients b_{kp} . Hence, variables A and B determine the cutting pattern that is generated. The objective function (11) is related to the pricing formula in (10). Constraints (12) assures that the solution of the model represents a valid cutting pattern for the BPPS. Notice that W is the capacity of a bin, then if a scenario $k \in \mathcal{K}$ is used ($B_k = 1$), the sum of the sizes of all items that belong to the scenario k must

be at most W , and if this scenario is not used ($B_k = 0$), then the sum of the sizes of all items that belong to k must be equal to 0.

7.3 The branch-and-price algorithm

To solve the model (5)-(9), we propose a branch-and-price algorithm. The branch-and-price is based on an enumeration of the fractional solution of the column generation algorithm. For this end, we use the branching scheme of Ryan and Foster (1981). Given a fractional solution X^* , we know, from Vance et al. (1994), that there exists rows l and m such that

$$0 < \sum_{p: a_{lp}=1, a_{mp}=1} X_p < 1.$$

We use this pair of rows to create two branches for the current node. On one side we enforce that items l and m must be packed in the same pattern, and on the other side we enforce that they must be packed in different patterns. This is achieved by the following branching constraints

$$\sum_{p: a_{lp}=1, a_{mp}=1} X_p \geq 1 \quad (14)$$

$$\sum_{p: a_{lp}=1, a_{mp}=1} X_p \leq 1 \quad (15)$$

Rather than explicitly adding these constraints to the RMP, they are enforced by setting the infeasible columns' upper bound to zero. On the branch of (14), this is equivalent to combine items l and m . For the branch of (15), this is equivalent to making rows l and m disjoint. However, this is not enough, since we must also avoid infeasible columns to be re-generated on each branch, which can be achieved by modifying the underlying pricing subproblem as follows:

- For the branches in which items l and m are combined, it suffices to modify the set of items to $\tilde{\mathcal{I}} = \{1, \dots, n\} \setminus \{l, m\} \cup \{n+1\}$, with $\alpha_{n+1} = \alpha_l + \alpha_m$,

$$s_{n+1}^k = \begin{cases} s_l^k, & \text{if } s_l^k > 0 \text{ and } s_m^k = 0, \\ s_m^k, & \text{if } s_l^k = 0 \text{ and } s_m^k > 0, \\ s_l^k + s_m^k, & \text{if } s_l^k > 0 \text{ and } s_m^k > 0, \\ 0, & \text{otherwise;} \end{cases} \quad (16)$$

- For branches in which items l and m must be packed in different patterns, we simply add the following disjunction constraint

$$A_l + A_m \leq 1. \quad (17)$$

These changes can be easily accommodated into the model (11)–(13) without much effort.

To obtain an upper bound at each node, we solve the current restricted master problem with the integrality constraint. In other words, we solve an ILP model that consists only of the columns that have been already generated. The use of this method at each node can be computationally expensive, but it usually provides tight upper bounds.

Some instances of the problem may be too complex to be solved by the column generation algorithm in practical time. In this manner, we would need faster methods for obtaining lower bounds for such instances. Two methods are proposed to get a lower bound at the root node of the branch-and-price algorithm. One of them is based on the continuous lower bound for the bin packing problem, and it is given by the following equation:

$$LB_{CON} = \max_{k \in \mathcal{K}} \left\{ \left\lceil \sum_{i \in S_k} s_i^k / W \right\rceil \right\}. \quad (18)$$

The idea behind the continuous lower bound (18) is to break items into unitary-sized items to fill empty spaces in a packing, which could not be filled otherwise. The LB_{CON} computes the continuous lower bound for every scenario, and returns the largest value between them.

The other lower bound that we use is based on dual feasible functions. A *dual feasible function* (DFF) is a function f such that, for any finite set X of real numbers, if $\sum_{x \in X} x \leq 1$, then $\sum_{x \in X} f(x) \leq 1$. For a survey on DFFs, we refer to Clautiaux et al. (2010). Given a DFF f , we can construct a lower bound based on it, similarly to the continuous lower bound, as given in the following equation:

$$LB_{DFF} = \max_{k \in \mathcal{K}} \left\{ \left\lceil \sum_{i \in S_k} f(s_i^k / W) \right\rceil \right\}. \quad (19)$$

In the proposed branch-and-price algorithm, given $\lambda \in \{1, \dots, W/2\}$, we consider the DFF proposed by Fekete and Schepers (2001) and described in (20):

$$f(s) = \begin{cases} W, & \text{if } s > W - \lambda; \\ 0, & \text{if } s \leq \lambda; \\ s, & \text{otherwise.} \end{cases} \quad (20)$$

In the experiments of Sect. 8, we test $\lambda = W - s_i$ for all $i \in \mathcal{I}$ and choose the one that maximizes LB_{DFF} for each instance.

8 Numerical Experiments

In this section, we evaluate the proposed VNS and B&P algorithms with computational experiments. Both solutions methods were implemented in the C++ programming language, with the B&P algorithm using the Gurobi Solver version 8.1 Gurobi Optimization (2023) to solve the integer linear programming (pricing) and linear programming (RMP) models. The experiments were carried out in a computer with an Intel(R) Xeon(R) CPU E5-2630 v4 processor at 2.20 GHz, with 64 GB of RAM and under the Ubuntu 18.04 operating system.

We generated a total of 120 instances divided into 12 different classes of 10 instances each. The classes are combinations of the number of items $n \in \{10, 50, 100, 200\}$ and the number of scenarios $d \in \{0.5n, n, 2n\}$, which depends on the number of items of the class. For all instances, the size of the items is randomly generated over the set $\{1, \dots, 99\}$ under a uniform probability distribution, whereas the size of the bin is fixed to 100. The scenarios were generated randomly, similarly to Bódis and Balogh (2019), such that an item $i \in \mathcal{I}$ belongs to the scenario $k \in \mathcal{K}$ with probability 0.5. For future experimentation, these instances were made available online on our public repository.¹

Since our exact methods are able to find lower bounds for all the proposed instances, we also use these bounds to evaluate the quality of the solutions provided by the VNS algorithm. We start by analyzing the performance of the VNS algorithm as well as the FFD algorithm used as its initial solution strategy. For these tests, we consider a time limit of 1800 seconds and limit of iterations without improving of 500. Table 1 describes the results of these experiments. Columns n and d present, respectively, the number of items and the number of scenarios of each class of instances. For VNS, columns *gap* represent the average optimality gap achieved in each class, computed as $(UB - LB)/UB$, where UB is the upper bound given by VNS and LB is the lower bound obtained by the exact methods. Columns *time* represent the average time in seconds. Columns $|OPT|$ represent the number of instances from each class that were solved to optimality (out of 10). For FFD, column gap_{vns} represents the average of the gap between FFD and VNS solutions in each class. Column *gap* is analogous to the one for VNS.

The VNS algorithm has no difficulty finding optimal solutions for most instances with 10 items. However, for instances of size 50 and 100 we notice a significant larger optimality gap, even though the time limit is never reached. This indicates that the VNS quickly found a local-optima and was unable to escape it, thus prematurely converging. Something similar can be observed for the instances of size 200, although the running time gets closer to the time limit in these cases. This algorithm, was able to solve 31 out of 120 instances to optimality, only two of them having size greater than 10. It was able to achieve a total average optimality gap of 8.32% in an average time of 390.01 seconds.

We also evaluate the performance of our implementation of the FFD heuristic in order to assess how much the VNS algorithm was able to improve its initial solution. To achieve this, we compute the gap between the value of the VNS solution (UB_{vns}) and

¹ <https://gitlab.com/glebbyo/algorithms-for-bpp-scenarios>

Table 1 Summary of the computational results for VNS and FFD

<i>n</i>	<i>d</i>	VNS			FFD	
		<i>gap</i>	<i>time</i>	$ OPT $	<i>gap_{vns}</i>	<i>gap</i>
10	5	0.00%	0.00	10	5.83%	5.83%
10	10	0.00%	0.00	10	2.50%	2.50%
10	20	1.67%	0.00	9	4.76%	6.19%
50	25	7.11%	14.80	1	8.30%	14.81%
50	50	5.24%	28.80	1	11.39%	16.06%
50	50	7.26%	28.80	0	8.28%	14.93%
100	200	9.34%	134.30	0	11.84%	20.06%
100	50	8.25%	181.20	0	9.61%	17.08%
100	200	7.78%	240.40	0	7.59%	14.78%
200	100	16.32%	1048.80	0	9.30%	24.15%
200	200	18.56%	1407.70	0	8.42%	25.45%
200	400	18.31%	1595.30	0	6.97%	24.02%
Average		8.32%	390.01	2.583	7.90%	15.49%

the value of the FFD solution (UB_{FFD}) as $gap_{vns} = (UB_{FFD} - UB_{VNS})/UB_{FFD}$. The average of the gap_{vns} of the instances of each class is reported in Table 1. The FFD algorithm was able to find optimal solutions only for a few instances with 10 items, which are the only cases for which VNS was unable to improve the solution given by FFD. In fact, the gap between the two algorithms increases significantly for instances of size 50, 100 and 200. On average, the VNS algorithm was able to improve the FFD solution by 7.90%.

To evaluate the exact model we test two different configurations: the pure B&P algorithm with a time limit of 3600 seconds; and B&P algorithm using the VNS solution as a warm-start, with a time limit of 1800 seconds. We use 1800 seconds on the latter because we consider that the VNS would take up to 1800 seconds in the warm-start phase. Table 2 summarizes the results for these experiments. We consider two sets of columns, each representing one of the tested configurations. Columns *gap* represent the average optimality gap that each algorithm achieved in each class, computed as $(UB - LB)/UB$, where UB and LB are respectively the upper and lower bounds obtained. Columns *time* represent the average time in seconds. Columns $|OPT|$ represent the number of instances from each class that were solved to optimality (out of 10) by each algorithm. Columns *cols* and *nodes* represent the average number of columns generated and nodes explored, respectively.

We notice a significant improvement in these experiments. All instances of 10 and 50 items are solved to optimality very quickly (100s on average). We also notice a drop in performance for instances of size 100 and 200. For instances of size 100, this algorithm was able to solve 11 instances to optimality, out of 30, averaging over 1800 seconds for all number of scenarios. We can also observe that the average gap for these instances is very small (around 2% for all number of scenarios). The algorithm reached the time limit for all instances of size 200, with an average gap of 14%. Interestingly, the average gaps achieved by the B&P for these instances are not far from the ones

Table 2 Summary of the computational results for B&P and B&P + VNS

<i>n</i>	<i>d</i>	B&P				VNS + B&P					
		<i>gap</i>	<i>nodes</i>	<i>cols</i>	<i>time</i>	$ OPT $	<i>gap</i>	<i>nodes</i>	<i>cols</i>	<i>time</i>	$ OPT $
10	5	0.00%	1	6.9	0.02	10	0.00%	0	0	0.00	10
10	10	0.00%	1.6	14.0	0.03	10	0.00%	0.3	5.4	0.00	10
10	20	0.00%	1.2	11.5	0.03	10	0.00%	0.3	3.4	0.00	10
50	25	0.00%	6.4	156.1	11.21	10	1.01%	955.8	98256.4	364.99	8
50	50	0.00%	15.7	289.8	30.08	10	0.50%	321.8	34501.3	187.00	9
50	50	0.00%	270.2	2401.1	569.41	10	0.93%	129.8	10871.7	370.18	8
100	200	1.44%	80.8	2381.6	1832.76	5	0.56%	6.6	4575.5	516.59	8
100	50	2.21%	85.3	2399.6	3019.09	2	1.38%	29.2	12419.1	1193.98	5
100	200	1.51%	41.4	1028.1	2710.84	4	1.72%	5.8	3577.8	1052.04	6
200	100	13.74%	1	532.7	3814.57	0	20.13%	1	3323.1	1817.98	0
200	200	17.52%	1	460.3	3925.27	0	20.07%	1	4801.9	1828.71	0
200	400	20.16%	1	362.7	3866.15	0	20.29%	1	4832.4	1817.93	0
Average		4.71%	42.21	837.03	1648.28	5.91	5.55%	121.05	14764	762.45	6.16

obtained by the VNS, with the latter having half the time limit of the former. The total average gap for this algorithm is 4.71% in an average time of 1648.28 seconds.

Finally, we evaluate B&P using the VNS solution as a warm-start (VNS + B&P). Once again, all instances of size 10 are completely solved in negligible time. Differing from the pure B&P algorithm though, it could not find optimal solutions for 5 instances of size 50. These instances brought all the averages for this group of instances up, since they were tried until the time limit of 1800 seconds, totaling thousands of columns and hundreds of nodes more than their non-warm-started counterpart. This happens because the solution of the VNS algorithm might impact on branching choices of the B&P, making it harder for the algorithm to converge depending on the branch it takes initially. However, we notice that for instances of size 100, VNS + B&P solved 19 instances to optimality, 8 more than the pure B&P and with a significant smaller average time. In this configuration, the algorithm also timed out for all instances of size 200, while presenting a greater gap in average compared to the pure B&P because of the smaller time limit. When compared to the pure B&P algorithm, VNS + B&P gives an average gap greater by 0.16% while using less than half the time in average.

The data from Table 1 and Table 2 show that although VNS struggles to avoid local-optima, its results serve as good upper bounds to be fed into more elaborated models such as the B&P presented in this work. By using the VNS solution as a warm-start, B&P was able to solve more instances in less time, though this was not enough to tackle instances with 200 items.

9 Concluding Remarks

This work deals with a variant of the bin packing problem where items occupy the bin capacity when they belong to the same scenario. For this problem, a variety of solution methods are proposed, including approximation algorithms, an asymptotic polynomial time approximation scheme, a variable neighborhood search based heuristic, and a branch-and-price that solves a set-cover-based formulation.

The contributions of this work nicely complement each other: Approximation algorithms offer theoretical guarantees, but they have shortcomings of either needing an approximation algorithm for the VBPP or restricting the problem to a fixed number of scenarios; On the other hand, practical contexts can be effectively addressed with the heuristic and exact methods proposed.

Section 4 derives constant-factor approximations for BPPS by reducing the problem to a particular case of VBPP, where non-zero coordinates of each input vector have the same value. Whilst this establishes some interesting connection between BPPS and VBPP, the reduction works only in one direction, which suggests that VBPP might be harder to approximate than BPPS. Indeed, this is true for the case with constant d , for which Sect. 5 gives an APTAS. This result contrasts with the hardness of VBPP, for which there is no APTAS even for $d = 2$ unless $P = NP$ (Woeginger 1997; Ray 2024).

Numerical experiments show that the VNS heuristic is not so competitive with the branch-and-price algorithm, since the former only optimally solved 26% of the instances, while the latter obtained optimal solutions for 59% of the instances. How-

ever, if solutions of the heuristic are used as a warm-start for the branch-and-price, the number of optimal solutions increases to about 62%. In total, we were able to prove optimality for 79 out of the 120 proposed instances.

Future research can focus on new approximation algorithms that explore the influence of the scenarios. Improvements in the variable neighborhood search are also expected, including new neighborhood structures and local searches. Regarding the branch-and-price, one could work on valid cuts as well as faster tailor-made algorithms for the pricing problem.

Funding The authors acknowledge the financial support of the National Council of Technological and Scientific Development (CNPq grants 312271/2023–9, 315555/2023–8, 312345/2023–2, 404315/2023–2, 408722/2023–1, 313146/2022–5, 405369/2021–2, 144257/2019–0), the State of Goiás Research Foundation (FAPEG), and the State of São Paulo Research Foundation (FAPESP grants 2022/05803-3, 2017/11831–1). This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

Data availability The instances generated are available in <https://gitlab.com/glebbyo/algorithms-for-bpp-scenarios>.

Declarations

Conflict of interest Author Yulle G. F. Borges declares that he has no conflict of interest. Author Vinícius L. de Lima declares that he has no Conflict of interest. Author Flávio K. Miyazawa declares that he has no Conflict of interest. Author Lehlilton L. C. Pedrosa declares that he has no Conflict of interest. Author Thiago A. de Queiroz declares that he has no Conflict of interest. Author Rafael C. S. Schouery declares that he has no Conflict of interest.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

References

- Alves C, ValÁrio de Carvalho J, Clautiaux F, Rietz J (2014) Multidimensional dual-feasible functions and fast lower bounds for the vector packing problem. *Eur J Oper Res* 233(1):43–63
- Azar Y, Cohen IR, Kamara S, Shepherd FB (2013) Tight bounds for online vector bin packing. In: Boneh D, Roughgarden T, Feigenbaum J (eds) *Symposium on Theory of Computing Conference, STOC'13*, Palo Alto, CA, USA, June 1–4, 2013, ACM, pp 961–970, <https://doi.org/10.1145/2488608.2488730>,
- Baghalian A, Rezapour S, Farahani RZ (2013) Robust supply chain network design with service level against disruptions and demand uncertainties: a real-life case. *Eur J Oper Res* 227(1):199–215
- Baldi MM, Manerba D, Perboli G, Tadei R (2019) A generalized bin packing problem for parcel delivery in last-mile logistics. *Eur J Oper Res* 274(3):990–999
- Balogh J, Bekesi J, Dósa G, Sgall J, van Stee R (2019) The optimal absolute ratio for online bin packing. *J Comput Syst Sci* 102:1–17
- Bansal N, Caprara A, Sviridenko M (2009) A new approximation method for set covering problems, with applications to multidimensional bin packing. *SIAM J Comput* 39(4):1256–1278. <https://doi.org/10.1137/080736831>
- Beltrán JD, Calderón JE, Cabrera RJ, Moreno-Pérez JA, Moreno-Vega JM (2004) Grasp-vns hybrid for the strip packing problem. *Hybrid Metaheuristics* 2004:79–90
- Bertazzi L, Golden B, Wang X (2019) The bin packing problem with item fragmentation: a worst-case analysis. *Discret Appl Math* 261:63–77
- Bettinelli A, Ceselli A, Righini G (2010) A branch-and-price algorithm for the variable size bin packing problem with minimum filling constraint. *Ann Oper Res* 179(1):221–241

- Bódis A, Balogh J (2019) Bin packing problem with scenarios. *Central Eur J Oper Res* 27(2):377–395. <https://doi.org/10.1007/s10100-018-0574-3>
- Borges YG, Miyazawa FK, Schouery RC, Xavier EC (2020) Exact algorithms for class-constrained packing problems. *Comput Ind Eng* 144:106455
- Brandao F, Pedroso JP (2016) Bin packing and related problems: general arc-flow formulation with graph compression. *Comput Oper Res* 69:56–67
- Buljubašić M, Vasequez M (2016) Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. *Comput Oper Res* 76:12–21
- Caprara A, Toth P (2001) Lower bounds and algorithms for the 2-dimensional vector packing problem. *Discret Appl Math* 111(3):231–262
- Chekuri C, Khanna S (2004) On multidimensional packing problems. *SIAM J Comput* 33(4):837–851
- Christensen HI, Khan A, Pokutta S, Tetali P (2017) Approximation and online algorithms for multidimensional bin packing: A survey. *Comput Sci Rev* 24:63–79. <https://doi.org/10.1016/j.cosrev.2016.12.001>
- Clautiaux F, Alves C, Valério de Carvalho J (2010) A survey of dual-feasible and superadditive functions. *Ann Oper Res* 179(1):317–342
- Clautiaux F, Dell’Amico M, Iori M, Khanafer A (2014) Lower and upper bounds for the bin packing problem with fragile objects. *Discret Appl Math* 163:73–86
- Coffman EG, Csirik J, Galambos G, Martello S, Vigo D (2013) Bin packing approximation algorithms: survey and classification. *Handbook of combinatorial optimization*. Springer, Cham, pp 455–531
- Dahmani N, Clautiaux F, Krichen S, Talbi EG (2013) Iterative approaches for solving a multi-objective 2-dimensional vector packing problem. *Comput Ind Eng* 66(1):158–170
- Dantzig G, Wolfe P (1961) The decomposition algorithm for linear programs. *Econometrica* 29(4):767–778
- Dell’Amico M, Furini F, Iori M (2020) A branch-and-price algorithm for the temporal bin packing problem. *Comput Oper Res* 114:104825
- Delorme M, Iori M (2019) Enhanced pseudo-polynomial formulations for bin packing and cutting stock problems. *INFORMS J Comput* 32(1):101–119
- Delorme M, Iori M, Martello S (2016) Bin packing and cutting stock problems: mathematical models and exact algorithms. *Eur J Oper Res* 255(1):1–20
- Fekete SP, Schepers J (2001) New classes of fast lower bounds for bin packing problems. *Math Program* 91(1):11–31
- Fernandez de la Vega W, Lueker GS (1981) Bin packing can be solved within $1 + \epsilon$ in linear time. *Combinatorica* 1(4):349–355
- Feuerstein E, Marchetti-Spaccamela A, Schalekamp F, Sitters R, van der Ster S, Stougie L, van Zuylen A (2014) Scheduling over scenarios on two machines. In: *International Computing and Combinatorics Conference*, pp 559–571
- Fleszar K, Hindi KS (2002) New heuristics for one-dimensional bin-packing. *Comput Oper Res* 29(7):821–839
- Ford L, Fulkerson D (1958) A suggested computation for maximal multi-commodity network flows. *Manage Sci* 5(1):97–101
- Garey MR, Johnson DS (1978) “Strong” NP-completeness results: motivation, examples, and implications. *J ACM (JACM)* 25(3):499–508
- Gilmore P, Gomory R (1961) A linear programming approach to the cutting stock problem. *Oper Res* 9:849–859
- Gilmore P, Gomory R (1963) A linear programming approach to the cutting stock problem - part II. *Oper Res* 11:863–888
- Grange A, Kacem I, Martin S (2018) Algorithms for the bin packing problem with overlapping items. *Comput Ind Eng* 115:331–341
- Gurobi Optimization L (2023) Gurobi optimizer reference manual. <http://www.gurobi.com>
- Heßler K, Gschwind T, Irnich S (2018) Stabilized branch-and-price algorithms for vector packing problems. *Eur J Oper Res* 271(2):401–419
- Hu Q, Zhu W, Qin H, Lim A (2017) A branch-and-price algorithm for the two-dimensional vector packing problem with piecewise linear cost function. *Eur J Oper Res* 260(1):70–80
- Juan AA, Kelton WD, Currie CS, Faulin J (2018) Simheuristics applications: dealing with uncertainty in logistics, transportation, and other supply chain areas. In: *2018 winter simulation conference (WSC)*, pp 3048–3059

- Kantorovich LV (1960) Mathematical methods of organizing and planning production. *Manag Sci* 6:363–422 (in Russian 1939)
- Kucukyilmaz T, Kiziloz HE (2018) Cooperative parallel grouping genetic algorithm for the one-dimensional bin packing problem. *Comput Ind Eng* 125:157–170
- Lee CC, Lee DT (1985) A simple on-line bin-packing algorithm. *J ACM* 32(3):562–572. <https://doi.org/10.1145/3828.3833>
- de Lima VL, Iori M, Miyazawa FK (2023) Exact solution of network flow models with strong relaxations. *Math Program* 197(2):813–846. <https://doi.org/10.1007/s10107-022-01785-9>
- Mladenović N, Hansen P (1997) Variable neighborhood search. *Comput Oper Res* 24(11):1097–1100
- Pereira J (2016) Procedures for the bin packing problem with precedence constraints. *Eur J Oper Res* 250(3):794–806
- Puchinger J, Raidl GR (2008) Bringing order into the neighborhoods: relaxation guided variable neighborhood search. *J Heuristics* 14(5):457–472
- Ray A (2024) There is no aptas for 2-dimensional vector bin packing: Revisited. *Inf Process Lett* 183:106430. <https://doi.org/10.1016/j.ipl.2023.106430>
- Ryan DM, Foster BA (1981) An integer programming approach to scheduling. *Computer scheduling of public transport urban passenger vehicle and crew scheduling* pp 269–280
- Sadykov R, Vanderbeck F (2013) Bin packing with conflicts: a generic branch-and-price algorithm. *INFORMS J Comput* 25(2):244–255
- Saint-Guillain M, Paquay C, Limbourg S (2021) Time-dependent stochastic vehicle routing problem with random requests: application to online police patrol management in brussels. *Eur J Oper Res* 292(3):869–885
- Sweeney PE, Paternoster ER (1992) Cutting and packing problems: a categorized, application-orientated research bibliography. *J Oper Res Soci* 43(7):691–706
- Vance PH, Barnhart C, Johnson EL, Nemhauser GL (1994) Solving binary cutting stock problems by column generation and branch-and-bound. *Comput Optim Appl* 3(2):111–130
- Vazirani V (2001) *Approximation Algorithms*. Springer-Verlag, New York
- Wäscher G, Haussner H, Schumann H (2007) An improved typology of cutting and packing problems. *Eur J Oper Res* 183(3):1109–1130
- Wei L, Lai M, Lim A, Hu Q (2020) A branch-and-price algorithm for the two-dimensional vector packing problem. *Eur J Oper Res* 281(1):25–35
- Wei L, Luo Z, Baldacci R, Lim A (2020) A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems. *INFORMS J Comput* 32(2):428–443
- Woeginger GJ (1997) There is no asymptotic ptas for two-dimensional vector packing. *Inf Process Lett* 64(6):293–297
- Xy Xu, Liu J, Hy Li, Jiang M (2016) Capacity-oriented passenger flow control under uncertain demand: algorithm development and real-world case study. *Transp Res Part E Logist Transp Rev* 87:130–148

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.